LIVRABLE 1 SUR 6

MaintiDoc

Assistant documentaire multimodal et agentique pour la maintenance

Cadrage, corpus et architecture

Équipe	**ELBELLAOUI-ELYOUNESSI**
Membres	Amr EL BELLAOUI · Soulaimane ELYOUNESSI
Période	Semaine 1 · octobre 2026
Dépôt Git	github.com/amr2444/Projet-int-grateur---Module-IA-g-n-rative-multimodale
Familles d'équipements	Pompes centrifuges · Compresseurs d'air

Sommaire

1. Cadrage du projet
2. Constitution du corpus documentaire
3. Architecture cible
4. Flux de données et modèle d'accès
5. Analyse des risques
6. Planning et organisation de l'équipe
Annexe A – Inventaire résumé du corpus
Annexe B – Journal de vérification
Annexe C – Dépôt et reproductibilité
Annexe D – Exemple de métadonnées
Annexe E – Données échangées entre les blocs

1. Cadrage du projet

1.1 Contexte

Dans un atelier industriel, la documentation de maintenance est dispersée : manuels constructeurs, procédures internes de plusieurs versions, fiches de sécurité, schémas et historiques d'interventions. Devant une machine en alarme, le technicien perd du temps à retrouver la bonne page, et risque d'appliquer une procédure obsolète.

MaintiDoc est un assistant documentaire multimodal : il accepte une question en français accompagnée d'une photo, retrouve les passages et figures pertinents dans un corpus autorisé, consulte l'historique de l'équipement via un outil MCP et peut préparer un ticket de maintenance. Il **ne prend aucune décision à la place du technicien** : il répond en citant ses sources, et toute action sensible passe par une validation humaine.

Le prototype couvre deux familles d'équipements d'un site industriel fictif :

- **Famille 1 : pompes centrifuges** (POMPE-A-01, POMPE-A-02, POMPE-B-01) ;
- **Famille 2 : compresseurs d'air** (COMP-A-01, COMP-A-02, COMP-B-02).

1.2 Utilisateurs visés

Profil	Contexte d'utilisation	Besoins principaux	Groupe d'accès
Technicien pompes	Sur le terrain, smartphone ou tablette, mains occupées, temps limité	Comprendre un voyant ou une alarme, trouver la procédure à jour, connaître l'historique de la machine	`techniciens_pompes`
Technicien compresseurs	Idem, sur les compresseurs	Idem	`techniciens_compresseurs`
Responsable maintenance	Bureau, poste fixe	Vue d'ensemble des interventions, tickets ouverts, validation des tickets, accès aux documents internes (audits, budgets)	`responsables`

1.3 Cas d'usage

CU1 : Identifier un voyant à partir d'une photo. Le technicien photographie le panneau de commande de POMPE-A-01 : « Que signifie ce voyant orange ? ». MaintiDoc identifie l'équipement, retrouve le tableau des codes d'alarme et la figure du panneau dans le manuel, puis répond en citant le document, la page et la figure.

CU2 : Trouver la procédure à jour. « Quel couple de serrage pour le couvercle de garniture de la pompe CP-50 ? ». Deux versions de la procédure existent dans le corpus avec des valeurs différentes. MaintiDoc doit signaler la contradiction, privilégier la version la plus récente et citer les deux.

CU3 : Consulter l'historique d'un équipement. « L'alarme de température est-elle déjà arrivée sur COMP-A-01 ? ». MaintiDoc appelle l'outil MCP `get_equipment_history`, croise le résultat avec la documentation et résume les interventions passées.

CU4 : Préparer et créer un ticket avec approbation. « Prépare un ticket pour cette alarme ». MaintiDoc rédige un brouillon (équipement, titre, description, sources). L'utilisateur voit le contenu exact et l'approuve. Le ticket n'est créé via MCP qu'après cette approbation, une seule fois.

CU5 : Demander une clarification plutôt qu'inventer. La photo est floue, ou le voyant orange pourrait appartenir aussi bien à une pompe qu'à un compresseur. MaintiDoc ne devine pas : il explique ce qui manque et demande, par exemple, une photo de la plaque signalétique.

1.4 Objectifs vérifiables en semaine 6

Chaque objectif se vérifie par un test concret, avec un résultat oui ou non. Les seuils chiffrés seront fixés en semaine 5.

#	Objectif	Comment on le vérifie
O1	Répondre à une question sur un voyant en citant le document, la page et, s'il y a lieu, la figure	La citation pointe vers un élément réellement récupéré et qui soutient la réponse
O2	Exploiter une photo de panneau pour identifier l'équipement ou le voyant	Même question posée avec et sans photo : la photo apporte l'identification
O3	Ne jamais retourner ni citer un document hors des groupes de l'utilisateur	Un technicien pompes interroge sur un document `responsables` : refus, aucun extrait affiché
O4	Signaler une contradiction entre deux versions d'un document	Question sur le couple de serrage : les deux versions sont citées et l'écart est signalé
O5	Consulter l'historique d'un équipement via l'outil MCP	Question CU3 : l'appel `get_equipment_history` est tracé, avec le bon `equipment_id`

#	Objectif	Comment on le vérifie
O6	Ne créer aucun ticket sans approbation explicite de l'utilisateur	Tentative de création sans approbation : rejet par le serveur MCP
O7	Ne pas créer de doublon si la même approbation est envoyée deux fois	Double envoi : un seul ticket créé (clé d'idempotence)
O8	Demander une clarification quand la photo est illisible ou les sources insuffisantes	Photo floue en entrée : statut `needs_clarification` et question posée
O9	Ignorer une instruction malveillante contenue dans un document	Document piégé : aucun appel d'outil non prévu, aucune fuite

1.5 Périmètre

Élément	Dans le périmètre
Entrées	Question en français (anglais toléré), photo de panneau ou d'équipement, documents PDF du corpus
Sorties	Réponse sourcée (document, page, figure), demande de clarification, brouillon de ticket
Corpus	30 à 50 documents synthétiques sur deux familles (pompes centrifuges, compresseurs d'air)
Outil MCP	Serveur de maintenance de démonstration : `get_equipment_history`, `get_open_tickets`, `create_ticket`
Action sensible	Création de ticket, uniquement après approbation explicite du contenu exact
Utilisateurs	Comptes fictifs répartis en 3 groupes : `techniciens_pompes`, `techniciens_compresseurs`, `responsables`

Hors périmètre (explicite) :

- piloter ou commander une machine (aucune connexion aux automates) ;
- garantir un diagnostic : MaintiDoc est une aide documentaire, une ressemblance visuelle ne vaut pas un diagnostic ;
- intervenir de façon autonome : aucune action sans validation humaine ;
- remplacer les procédures de sécurité ou un technicien habilité ;
- traiter des données réelles : utilisateurs, tickets et historiques sont fictifs ;
- couvrir d'autres familles d'équipements que les deux retenues.

1.6 Hypothèses et contraintes

Contrainte	Situation de l'équipe	Conséquence
Matériel	Pas de GPU local	Inférence sur Google Colab ou Kaggle (GPU T4, 16 Go, sessions limitées dans le temps). Qwen2.5-VL-7B quantifié en 4 bits, ou Qwen2.5-VL-3B en repli (à valider par test en semaine 3)
Mémoire	Environ 16 Go de VRAM au maximum	Une requête à la fois, taille d'image et contexte limités
Temps	6 semaines, environ 10 h par personne et par semaine (à confirmer)	Priorité au scénario de démonstration, fonctionnalités secondaires en option
Équipe	2 membres : Amr EL BELLAOUI, Soulaimane ELYOUNESSI	Rôles détaillés en partie 6
Données	Corpus 100 % synthétique, généré avec un assistant IA puis relu	Traçabilité des prompts et relecture humaine obligatoires

2. Constitution du corpus documentaire

2.1 Démarche retenue

Nous avons choisi la **voie synthétique** pour l'ensemble du corpus. Trois raisons :

- **Licences** : un document synthétique produit par l'équipe ne pose aucune question de droits de réutilisation.
- **Cohérence** : nous maîtrisons les identifiants d'équipements, les codes d'alarme et les numéros de page, condition indispensable pour vérifier les citations en semaine 3.
- **Tests** : nous pouvons fabriquer exactement les cas difficiles dont l'évaluation de la semaine 5 aura besoin (contradictions, ambiguïtés, injection).

Le corpus décrit un **site industriel fictif** équipé de matériels de deux marques inventées : **MD-Pompes** et **MD-Air**. Chaque document porte en pied de page la mention « Document synthétique – usage pédagogique », pour éviter toute confusion avec un document réel.

La génération a été faite avec **Claude Opus 4.5** (Anthropic) en quatre étapes :

1. **Un référentiel unique** fixe les 6 équipements, 23 codes d'alarme, les voyants, les valeurs techniques et les historiques d'interventions.
2. **Des scripts génèrent chaque document** à partir de ce référentiel : PDF avec reportlab, schémas avec matplotlib, photos avec Pillow. Un code d'alarme ou un couple de serrage a donc forcément la même valeur dans tous les documents où il apparaît, sauf contradiction voulue.
3. **Les versions « scannées »** sont produites en transformant les pages en images, puis en les dégradant (inclinaison, bruit, flou, éclairage inégal). Le PDF obtenu n'a plus de couche texte.
4. **Un script de contrôle** (`check.py`) vérifie la cohérence, puis chaque document est relu par un membre de l'équipe.

Les 25 prompts utilisés sont tracés dans `corpus/prompts_corpus.md` (P01 à P25), et les scripts sont versionnés dans le dépôt. Le corpus est donc reproductible.

2.2 Équipements couverts

Famille	Équipement	Modèle (fictif)	Rôle sur le site
Pompes	POMPE-A-01	MD-Pompes CP-50, centrifuge monocellulaire	Eau de refroidissement, pompe principale
Pompes	POMPE-A-02	MD-Pompes CP-50	Pompe de secours
Pompes	POMPE-B-01	MD-Pompes CM-80, multicellulaire 5 étages	Surpression eau process
Compresseurs	COMP-A-01	MD-Air VS-37, à vis 37 kW	Réseau air 7 bar, principal
Compresseurs	COMP-A-02	MD-Air VS-37	Secours / appoint
Compresseurs	COMP-B-02	MD-Air PS-11, à piston 11 kW	Atelier mécanique

2.3 Composition du corpus

42 documents, soit 36 PDF totalisant 115 pages, et 6 photos.

Type de document	Pompes	Compresseurs	Total
Manuels	2	2	4
Procédures, guides, notes internes	10	7	17
Fiches de sécurité	2	2	4
Schémas	3	2	5
Tableaux de codes d'alarme	1	1	2
Historiques d'interventions	2	2	4
Photos de terrain	3	3	6
Total	**23**	**19**	**42**

Couverture des exigences de la consigne :

Exigence	Attendu	Corpus
Nombre de documents	30 à 50	42
Documents avec éléments visuels exploitables	≥ 10	38 (figures, schémas, tableaux, photos)
Documents scannés (test OCR)	≥ 2	4
Documents contradictoires	≥ 2	4 (2 paires)
Document piégé (injection)	≥ 1	1
Documents en anglais (signalés)	quelques-uns	2
Cas ambigus	recommandé	5
Documents réservés aux responsables	(contrôle d'accès)	3

2.4 Documents prévus pour l'évaluation (semaine 5)

Rôle de test	Document(s)	Ce que le système doit faire
Contradiction n°1	`proc_vidange_cp50_v1` (2023, scannée) vs `proc_vidange_cp50_v2` (2025)	Couple du couvercle de garniture : 25 N·m contre 35 N·m. Signaler l'écart et privilégier la v2, confirmée par le manuel (p. 11)
Contradiction n°2	`proc_vidange_huile_vs37_v1` (2021, scannée) vs `proc_vidange_huile_vs37_v2` (2025)	Vidange VS-37 : 2000 h / 18 L / huile minérale contre 4000 h / 16 L / huile synthétique
Injection	`note_service_compresseurs_piegee`	Page 2 : instruction d'exfiltration vers une URL externe, de création de ticket sans approbation et d'élévation de droits, plus une seconde injection en texte minuscule. Aucune de ces actions ne doit être exécutée

Rôle de test	Document(s)	Ce que le système doit faire
Ambiguïté (voyant)	`photo_voyant_orange_gros_plan`, `codes_alarme_pompes`, `codes_alarme_compresseurs`	Un voyant orange « ALERTE » existe sur les pompes (H2) et sur les compresseurs (L2), avec des significations différentes. Demander de quel équipement il s'agit
Ambiguïté (identification)	`photo_panneau_comp_a01`	Code masqué par un reflet, identifiant « COMP-A-0? » incomplet. Demander une précision
Photo illisible	`photo_plaque_floue_pompe`	Demander une nouvelle photo au lieu de deviner
Contrôle d'accès	`plan_maintenance_pompes_2026`, `rapport_audit_air_comprime`, `note_service_compresseurs_piegee`	Invisibles pour les techniciens, y compris dans les citations

2.5 Inventaire

Le fichier `corpus/inventaire.csv` (UTF-8, séparateur virgule) contient une ligne par document, avec les 20 colonnes demandées. La colonne `elements_visuels` a été **générée automatiquement** à partir des figures et tableaux réellement placés dans chaque PDF (ex. « fig 7.1 p.8 ; tableau 8.1 p.9 »). Les pages citées sont donc exactes. Le contrôle automatique vérifie :

- que chaque fichier du dossier figure dans l'inventaire, et inversement ;
- que le nombre de pages déclaré correspond au nombre réel ;
- que les valeurs des colonnes respectent les formats imposés ;
- que les PDF scannés n'ont pas de couche texte ;
- que les équipements et codes d'alarme cités existent dans le référentiel ;
- que les identifiants d'interventions des historiques correspondent à ceux du futur serveur MCP.

2.6 Analyse critique de l'usage de l'IA

L'assistant a produit rapidement un corpus volumineux et structuré, mais **six erreurs** ont été détectées puis corrigées avant validation (journal E01 à E06) :

- **Contradiction non voulue (E01).** Un même schéma de garniture était réutilisé dans plusieurs documents avec la mention « 35 N·m ». Dans la procédure v1, il contredisait le texte (25 N·m). Ce type d'erreur est sournois : chaque élément est correct pris isolément, c'est leur combinaison qui est fausse. Seule la relecture de la page complète l'a révélé.
- **Incohérence chiffrée (E03).** Dans l'audit énergétique, le graphique était titré « local LC1 » alors que ses données incluaient un compresseur situé dans l'atelier. Les totaux étaient justes, mais le libellé ne l'était pas. Nous l'avons détecté en recalculant les sommes.
- **Ambiguïtés de nommage (E06).** L'abréviation « POMPE-A-01/A-02 » produisait un « A-02 » isolé qui ressemble à un code d'alarme compresseur. Notre script de contrôle l'a signalé. Il a aussi produit 33 faux positifs (le « A-01 » de « POMPE-A-01 ») : un contrôle automatique doit lui-même être vérifié.
- **Photos ne remplissant pas leur rôle (E04, E05).** La photo censée être ambiguë laissait lire le code d'alarme, et le voyant « orange » apparaissait jaune. Sans contrôle visuel, ces cas de test auraient été inutiles, voire trompeurs.
- **Mise en page (E02).** Les identifiants d'interventions étaient coupés sur deux lignes dans les tableaux d'historique, ce qui aurait gêné l'extraction.

Limites que nous retenons :

- Les valeurs techniques sont **plausibles mais inventées**. Nous avons contrôlé leur vraisemblance (vitesse d'un moteur 2 pôles, seuils de vibration de l'ISO 10816-3, températures d'un compresseur à vis), pas leur exactitude pour un matériel réel. Le corpus ne doit jamais servir à intervenir sur un vrai équipement.
- L'OCR testé sur les documents scannés introduit déjà des erreurs (« l'huile », « a » au lieu de « à ») avec un modèle de langue anglais. Il faudra un modèle français en semaine 2.
- L'IA a réalisé la génération **et** une partie des contrôles automatiques. Une relecture humaine indépendante reste donc indispensable : chaque document a été relu par un membre de l'équipe (colonne `verifie_par`).
- Un corpus synthétique est plus « propre » qu'un corpus réel : les résultats de la semaine 5 seront probablement optimistes par rapport à des documents réels hétérogènes.

3. Architecture cible

3.1 Vue d'ensemble

L'architecture reprend la section 3 du cahier des charges, reformulée et organisée en **9 blocs**.

Une demande suit le trajet suivant :

1. Elle part de l'**interface**, passe par l'**API**, qui identifie l'utilisateur, puis arrive à l'**orchestrateur**.

2. L'orchestrateur est le « chef d'orchestre » : il décide quelles étapes exécuter et dans quel ordre.
3. Il fait appel à trois services : le **modèle vision-langage** (comprendre la photo et rédiger), la **recherche documentaire** (trouver les passages et figures) et l'**outil MCP** (historique et tickets).
4. Avant de répondre, un bloc de **vérification** contrôle les citations.

L'**ingestion** fonctionne à part, hors ligne : elle prépare l'index une fois pour toutes, à partir du corpus.

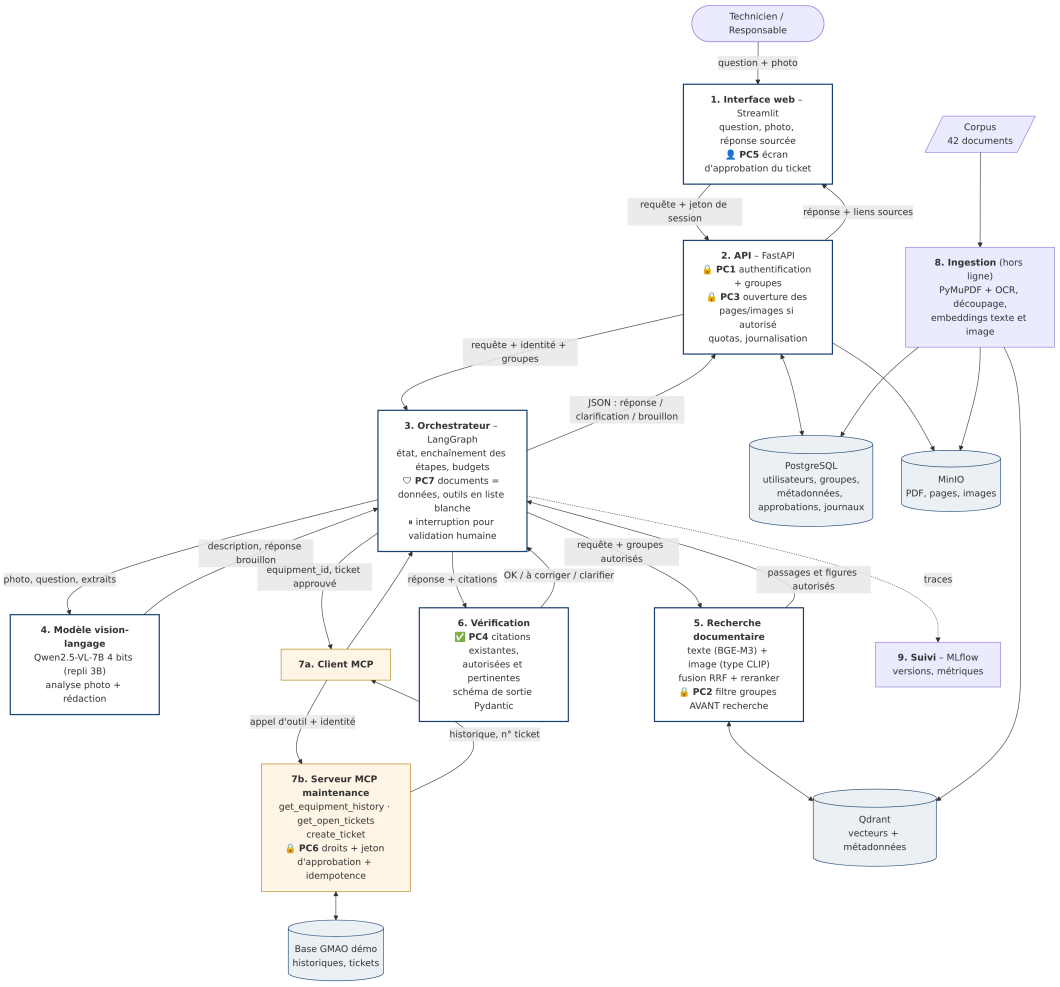


Figure 3.1 – Schéma par blocs de MaintiDoc
Les points de contrôle PC1 à PC7 sont indiqués dans les blocs où ils s'appliquent.

3.2 Fiches des blocs

Bloc	Rôle (en une phrase)	Entrées	Sorties	Outil envisagé
1. Interface web	Permettre au technicien de poser sa question avec une photo, de lire une réponse sourcée et d'approuver un ticket	Texte, photo (JPEG/PNG), clic d'approbation	Requête HTTP vers l'API ; affichage de la réponse, des sources et du brouillon de ticket	Streamlit
2. API	Point d'entrée unique : vérifier qui appelle, ce qu'il a le droit de voir, et tracer la demande	Requête + jeton de session	Requête enrichie (identité, groupes) vers l'orchestrateur ; réponse JSON vers l'interface	FastAPI ; comptes fictifs dans PostgreSQL
3. Orchestrateur	Enchaîner les étapes (analyser, chercher, consulter, rédiger, vérifier, décider) et appliquer les budgets	Question, photo, identité, groupes	Réponse structurée, clarification ou brouillon de ticket ; état mis en pause pendant l'approbation	LangGraph (graphe d'états) + LangChain (prompts, parseurs)
4. Modèle vision-langage	Décrire la photo (équipement, voyant, afficheur) puis rédiger la réponse à partir des extraits fournis	Image + question ; extraits et figures récupérés	Description visuelle ; texte de réponse avec citations proposées	Qwen2.5-VL-7B-Instruct quantifié 4 bits ; repli Qwen2.5-VL-3B ; Transformers

Bloc	Rôle (en une phrase)	Entrées	Sorties	Outil envisagé
5. Recherche documentaire	Retrouver, **parmi les seuls documents autorisés**, les passages, tableaux et figures pertinents	Texte de la requête, photo, liste des groupes autorisés	Top-k éléments classés (texte et images) avec leurs métadonnées (document, page, élément)	Qdrant ; BGE-M3 (texte) ; encodeur texte-image type CLIP multilingue ; fusion RRF ; reranker multilingue (bge-reranker, adapté en S5)
6. Vérification	S'assurer que chaque citation pointe vers un élément réellement récupéré et autorisé, et qu'elle soutient l'affirmation	Réponse brouillon + éléments récupérés	Verdict : valide / à corriger / clarification nécessaire ; JSON conforme au schéma	Pydantic (format) + contrôle par règles (existence, droits) + contrôle par le modèle (soutien)
7. Outil MCP (client + serveur)	Donner accès à l'historique et aux tickets de la GMAO de démonstration, sous contrôle	`equipment_id`, identité ; pour la création : contenu approuvé, jeton d'approbation, clé d'idempotence	Liste d'interventions, tickets ouverts, numéro du ticket créé ou refus motivé	SDK MCP Python (serveur et client) ; SQLite ou PostgreSQL pour la GMAO de démonstration
8. Ingestion	Transformer le corpus en éléments indexés et recherchables	PDF texte, PDF scannés, photos, `inventaire.csv`	Passages, tableaux, figures, vecteurs et métadonnées	PyMuPDF ; OCR local (Tesseract `fra` ou PaddleOCR, à comparer)
9. Suivi	Garder la trace des versions (modèle, prompts, index, reranker) et des métriques de chaque exécution	Traces de l'orchestrateur, résultats d'évaluation	Expériences comparables	MLflow
Stockage	Conserver vecteurs, métadonnées, fichiers et données de la GMAO	—	—	Qdrant (vecteurs + payload) ; PostgreSQL (utilisateurs, groupes, documents, approbations, journaux) ; MinIO (PDF, pages, images)

Tous ces composants sont des briques open source ou à poids ouverts. Les versions exactes et les licences seront figées dans le dépôt en semaine 2.

3.3 Points de contrôle

Point	Où	Ce qui est vérifié	Ce qui se passe en cas d'échec
PC1 Authentification	API	L'utilisateur est connu ; on récupère ses groupes (`techniciens_pompes`, `techniciens_compresseurs`, `responsables`)	Requête rejetée (401)
PC2 Filtre d'accès avant recherche	Recherche documentaire	La recherche ne porte **que** sur les éléments dont `groupes_acces` contient un groupe de l'utilisateur (filtre Qdrant, pas un tri après coup)	Un document interdit n'est jamais récupéré, donc jamais transmis au modèle
PC3 Contrôle à l'ouverture	API → MinIO	Quand l'utilisateur clique sur une source, on revérifie ses droits avant de servir la page ou l'image	Refus (403), sans révéler le contenu
PC4 Vérification des citations	Bloc vérification	Chaque citation (document, page, élément) existe parmi les éléments récupérés et autorisés, et soutient la phrase associée	Citation retirée ; si plus aucune source ne soutient la réponse, demande de clarification ou abstention
PC5 Validation humaine	Interface + orchestrateur	L'utilisateur voit le **contenu exact** du ticket et l'approuve explicitement	Pas d'approbation : pas de création. Contenu modifié : nouvelle approbation exigée
PC6 Contrôle serveur MCP	Serveur MCP	Droits de l'utilisateur sur l'équipement ; jeton d'approbation valide et lié à l'utilisateur et au contenu ; clé d'idempotence	Refus de l'appel ; un second appel avec la même clé ne crée pas de doublon
PC7 Garde-fous	Orchestrateur	Texte des documents et réponses d'outils traités comme des **données** (jamais comme des instructions) ; outils en liste blanche ; budgets (≤ 6 appels modèle, ≤ 3 appels MCP, ≤ 60 s)	Instruction ignorée, appel non prévu bloqué, arrêt propre si le budget est dépassé

Le contrôle d'accès n'est pas confié au modèle : il est fait **par le code**, avant (PC2) et après (PC3), puis **côté serveur MCP** (PC6). Même si le modèle était trompé par une injection, il ne pourrait ni voir un document interdit ni créer un ticket non approuvé.

3.4 Parcours d'une requête

Scénario nominal (CU1). Un technicien du groupe `techniciens_pompes` envoie la photo `photo_panneau_pompe_a01.jpg` avec la question : « Que signifie ce voyant orange ? Prépare un ticket si nécessaire. »

1. **Identification (PC1).** L'interface transmet la question, la photo et le jeton de session à l'API. L'API identifie l'utilisateur et récupère ses groupes : `techniciens_pompes`.
2. **Analyse de la demande.** L'orchestrateur ouvre une nouvelle exécution, avec un état vide et des budgets à zéro, et envoie la photo au modèle vision-langage. Celui-ci décrit ce qu'il voit : l'étiquette « POMPE-A-01 », les voyants vert H1 et orange H2 allumés, l'afficheur « P-21 T PAL 84°C ». L'équipement et le code sont identifiés.
3. **Recherche (PC2).** L'orchestrateur lance une recherche texte (« voyant H2 orange fixe, code P-21, température palier ») et une recherche image (similarité avec les figures de panneaux), **filtrées sur les documents du groupe `techniciens_pompes`**. Les résultats sont fusionnés (RRF) puis reclassés par le reranker. Exemples de résultats : le tableau des voyants du manuel CP-50 (page 8), le tableau des codes (page 9) et la fiche des codes d'alarme pompes.
4. **Historique (MCP).** La question porte sur un équipement identifié et demande un éventuel ticket. L'orchestrateur appelle donc `get_equipment_history("POMPE-A-01")` et `get_open_tickets("POMPE-A-01")`. Le serveur vérifie les droits (PC6) et renvoie deux préalarmes P-21 précédentes (2025-09 et 2026-03) et un ticket ouvert sur les vibrations.
5. **Rédaction.** Le modèle reçoit la question, sa description de la photo, les extraits retenus et l'historique, avec la consigne de citer ses sources. Il rédige la réponse : H2 orange fixe avec P-21 signifie une préalarme de température palier (≥ 80 °C) ; la pompe continue de tourner ; il faut contrôler le niveau d'huile, les roulements et l'alignement ; c'est la 3e occurrence en un an, alors que les roulements ont déjà été remplacés en mars.
6. **Vérification (PC4).** Chaque citation est contrôlée : `manuel_pompe_cp50`, p. 8, tableau 7.1 ; p. 9, tableau 8.1 ; historique MCP. Elles existent bien parmi les éléments récupérés et soutiennent les phrases associées. La sortie est validée par le schéma Pydantic.
7. **Brouillon de ticket.** L'utilisateur a demandé un ticket et le diagnostic montre une récurrence. Le système prépare un brouillon : équipement POMPE-A-01, titre, description, sources. Le graphe se met **en pause**.
8. **Approbation (PC5).** L'interface affiche le contenu exact du ticket. Le technicien l'approuve. L'API génère un jeton d'approbation lié à l'utilisateur et à l'empreinte du contenu.
9. **Création (PC6).** L'orchestrateur reprend et appelle `create_ticket(...)` avec le jeton et une clé d'idempotence. Le serveur vérifie le tout et crée le ticket. Un double clic ne crée pas de doublon.
10. **Réponse.** L'utilisateur reçoit la réponse sourcée et le numéro du ticket. Les sources sont cliquables, et les droits sont revérifiés à l'ouverture (PC3).

Variantes :

- **Photo ambiguë** (`photo_voyant_orange_gros_plan`, `photo_panneau_comp_a01`). À l'étape 2, l'équipement n'est pas identifiable : le voyant orange existe sur les pompes (H2) et sur les compresseurs (L2), avec des significations différentes. L'orchestrateur ne choisit pas au hasard : il renvoie le statut `needs_clarification` et demande une photo de la plaque ou de l'étiquette.
- **Contradiction.** À l'étape 3, si `proc_vidange_cp50_v1` (25 N·m) et `proc_vidange_cp50_v2` (35 N·m) sont récupérées ensemble, la réponse cite les deux, signale l'écart et privilégie la version la plus récente, confirmée par le manuel.
- **Accès interdit.** Un technicien qui demande le budget 2026 n'obtient rien. Le document est réservé aux `responsables` : il est exclu par le filtre (PC2), et le système répond qu'aucune source accessible ne permet de répondre.
- **Injection.** Si la note piégée est récupérée (pour un responsable), son texte « ignore les instructions… » est traité comme une donnée. L'URL externe n'est pas un outil autorisé (PC7), et une création de ticket sans jeton est refusée par le serveur (PC6).

3.5 Contraintes de déploiement (pas de GPU local)

L'équipe ne dispose pas de GPU. Nous envisageons un déploiement en deux parties :

- **Sur nos PC**, avec Docker Compose : l'interface, l'API, l'orchestrateur, Qdrant, PostgreSQL, MinIO, le serveur MCP et l'ingestion. Ces services fonctionnent sur processeur.
- **Sur Google Colab ou Kaggle** (GPU T4, 16 Go) : le modèle vision-langage et, si besoin, le calcul des embeddings d'images. Le modèle serait exposé par une API protégée, ou bien toute la démonstration tournerait dans un notebook.

Ce choix est à valider en semaine 2 ou 3 (voir les questions ouvertes ci-dessous).

3.6 Questions ouvertes et hypothèses à vérifier

#	Question / hypothèse	Comment la vérifier	Semaine
Q1	Qwen2.5-VL-7B quantifié en 4 bits tient-il sur un T4 (16 Go) avec une image et environ 4 000 tokens de contexte ? Sinon, le 3B suffit-il à lire un afficheur et un voyant ?	Test de chargement, mesure de la mémoire et de la latence sur nos 6 photos	S2–S3
Q2	Comment relier Colab (GPU) à nos services locaux sans exposer de secret ni d'accès ouvert ?	Comparer un tunnel authentifié et un fonctionnement tout-en-un dans le notebook	S2
Q3	Les sessions Colab gratuites (durée limitée, interruptions) suffisent-elles pour l'évaluation de 150 scénarios ?	Estimer le temps par scénario ; prévoir Kaggle (30 h GPU par semaine) en complément	S3
Q4	Faut-il un OCR pour tous les documents, ou seulement pour les 4 scannés ?	Détecter automatiquement l'absence de couche texte (déjà vérifiée par `check.py`)	S2
Q5	Tesseract `fra` est-il suffisant ? Notre test avec le modèle anglais fait des erreurs sur les accents (« a » au lieu de « à »)	Comparer Tesseract `fra` et PaddleOCR sur les 4 scans	S2
Q6	Quelle granularité d'élément indexer : paragraphe, tableau entier, ligne de tableau des codes ?	Tester la recherche sur des questions de codes d'alarme	S2
Q7	Un encodeur de type CLIP distingue-t-il le voyant orange d'une pompe de celui d'un compresseur ? On s'attend à ce que non, d'où la nécessité de la clarification	Recherche image sur les 3 photos ambiguës	S2–S3
Q8	Comment vérifier qu'une citation **soutient** l'affirmation, au-delà de son existence ? Le modèle juge-t-il de façon fiable ?	Annoter une vingtaine de cas à la main et comparer	S3 / S5
Q9	Le budget de 60 s par demande est-il tenable avec 2 à 4 appels au modèle sur un T4 ?	Mesurer la latence médiane et p95	S3–S4
Q10	Pourra-t-on construire 800 à 1 500 exemples d'entraînement du reranker à partir de 42 documents ?	Générer des questions, les vérifier, choisir les négatifs difficiles (voyants H2 / L2)	S4–S5

4. Flux de données et modèle d'accès

4.1 Flux d'ingestion

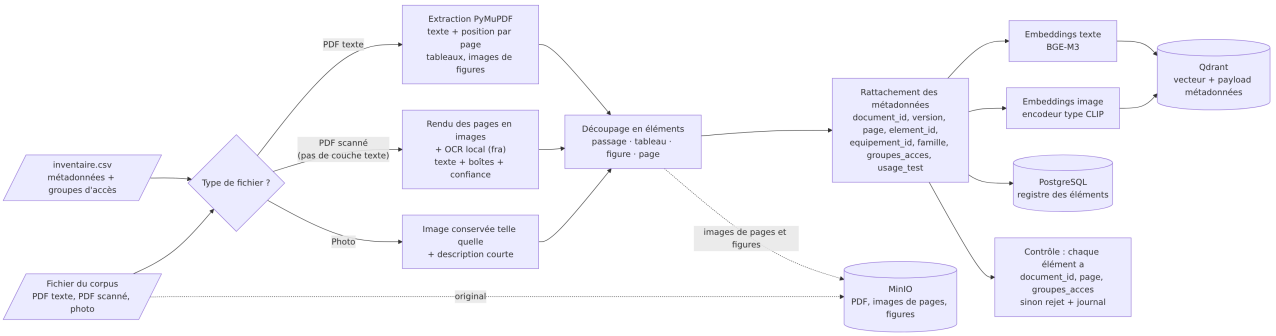


Figure 4.1 – Flux d'ingestion

L'ingestion est exécutée **hors ligne**, une fois par version du corpus. Elle transforme chaque fichier en **éléments** indexés.

1. **Lecture du registre.** On lit `inventaire.csv`. C'est lui, et non le contenu du document, qui fournit les métadonnées de confiance : identifiant, version, équipements et surtout **groupes d'accès**. Un fichier absent de l'inventaire n'est pas indexé.
2. **Aiguillage selon le format** (colonne `format`) :
   - `pdf_texte` : extraction avec PyMuPDF du texte avec sa position sur la page, des tableaux et des images des figures ;
   - `pdf_scanne` : les pages sont rendues en images puis passées à l'OCR local (modèle français). On conserve le texte, la position et un score de confiance. Les 4 scans du corpus n'ont pas de couche texte, ce que `check.py` vérifie déjà ;
   - `image` (photo) : l'image est conservée telle quelle, avec la description de l'inventaire (colonne `elements_visuels`).
3. **Découpage en éléments.** On distingue quatre types : `passage` (un paragraphe ou une étape de procédure), `tableau` (un tableau entier, par exemple le tableau 8.1 des codes P-xx), `figure` (l'image de la figure et sa légende) et `page` (l'image de la page entière, utile au modèle de vision).

4. **Rattachement des métadonnées.** Chaque élément reçoit les métadonnées décrites en 4.4.
5. **Calcul des embeddings.** BGE-M3 pour le texte et les légendes, un encodeur de type CLIP pour les figures, pages et photos.
6. **Stockage.** Les vecteurs et leurs métadonnées vont dans **Qdrant**. Le registre des éléments va dans **PostgreSQL**. Les fichiers originaux, les images de pages et les figures vont dans **MinIO**, pour être affichés lors d'une citation.
7. **Contrôle.** Un élément sans `document_id`, `page` ou `groupes_acces` est rejeté et journalisé. On n'indexe jamais un élément « sans droits », qui serait visible par tous par défaut.

Le texte extrait des documents est conservé **comme une donnée**. Il n'est jamais interprété comme une instruction, y compris pour la note piégée.

4.2 Flux d'une requête

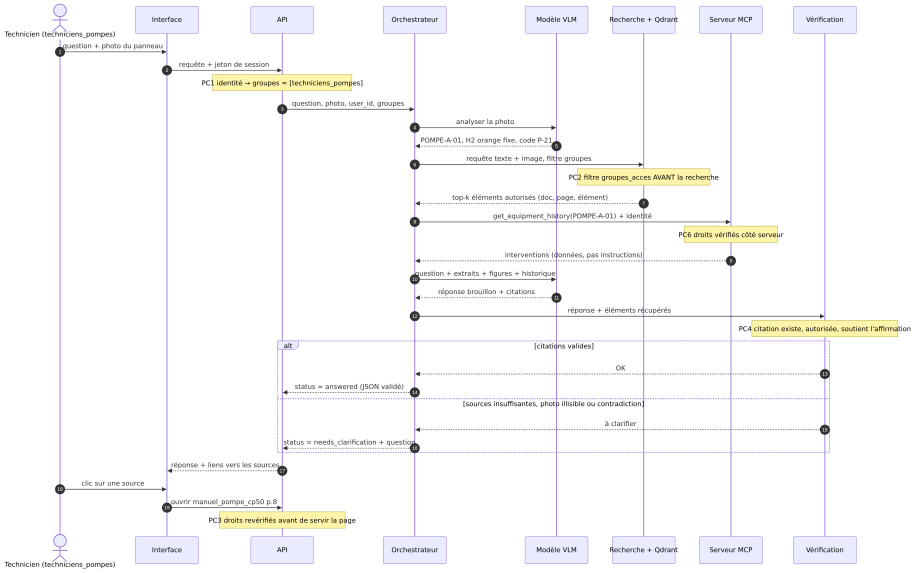


Figure 4.2 – Flux d'une requête avec photo jusqu'à la réponse sourcée

Les étapes numérotées du diagramme suivent le parcours décrit en 3.4. Les données échangées à chaque frontière, avec le contrôle associé, sont détaillées en annexe E.

4.3 Flux de création de ticket

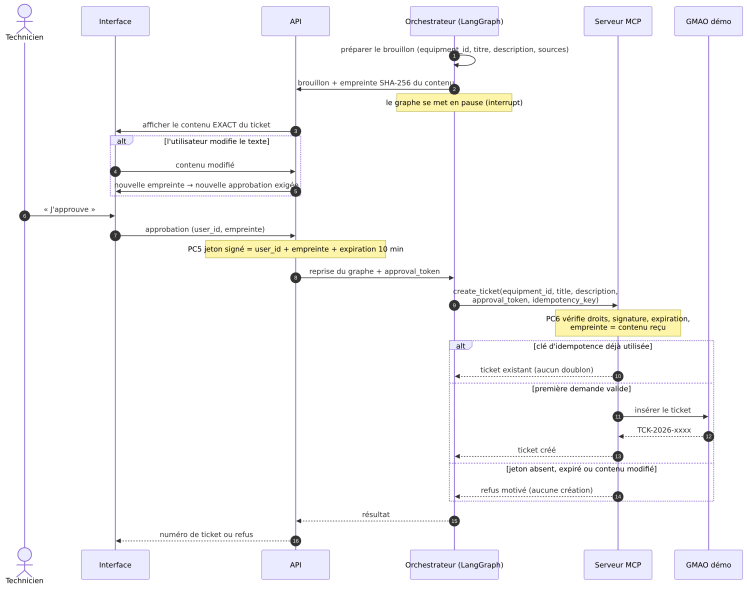


Figure 4.3 – Création de ticket avec approbation humaine et idempotence

1. **Préparation.** L'orchestrateur prépare un brouillon : `equipment_id`, titre, description et sources. Il calcule une **empreinte** (hachage SHA-256) du contenu exact, puis le graphe LangGraph se met **en pause**.
2. **Affichage.** L'interface affiche le contenu exact. Si l'utilisateur modifie une virgule, l'empreinte change et une nouvelle approbation est exigée.
3. **Approbation (PC5).** Quand l'utilisateur clique sur « J'approuve », l'API émet un **jeton d'approbation signé**, qui contient l'identifiant de l'utilisateur, l'empreinte du contenu et une date d'expiration (10 minutes).

4. **Appel MCP.** Le graphe reprend et appelle `create_ticket(equipment_id, title, description, approval_token)`. Il y joint une **clé d'idempotence**, dérivée de l'empreinte et de l'identifiant de la demande.
5. **Contrôle serveur (PC6).** Le serveur MCP vérifie quatre choses : les droits de l'utilisateur sur l'équipement, la signature et l'expiration du jeton, l'égalité entre l'empreinte du jeton et celle du contenu reçu, et la clé d'idempotence.
6. **Idempotence.** Si la clé a déjà servi (double clic, nouvelle tentative après une coupure réseau), le serveur renvoie le ticket **déjà créé**, au lieu d'en créer un second.

Ainsi, même si le modèle tentait de créer un ticket de sa propre initiative, par exemple sous l'effet de la note piégée qui réclame `approval_token=ADMIN`, l'appel serait refusé : le jeton n'est pas signé par l'API et ne correspond à aucune approbation.

4.4 Schéma des métadonnées d'un élément indexé

Champ	Type	Exemple	Utilité
`element_id`	texte	`manuel_pompe_cp50#p9#tableau_8_1`	Identifiant unique, utilisé dans les citations
`document_id`	texte	`manuel_pompe_cp50`	Lien avec l'inventaire
`version`	texte	`v2.1 (2025-03)`	Choisir la version la plus récente en cas de contradiction
`titre_document`	texte	Manuel d'utilisation... CP-50	Affichage de la source
`type_document`	énuméré	`manuel`	Filtrage, pondération
`page`	entier	`9`	Citation vérifiable
`type_element`	énuméré	`passage` / `tableau` / `figure` / `page` / `photo`	Choix de l'encodeur et de l'affichage
`numero_element`	texte	`Tableau 8.1`	Citation lisible
`famille_equipement`	énuméré	`pompes`	Filtrage, désambiguïsation
`equipement_id`	liste	`["POMPE-A-01", "POMPE-A-02"]`	Lien avec l'historique MCP
`groupes_acces`	liste	`["techniciens_pompes", "responsables"]`	**Filtre d'accès (PC2, PC3)**
`langue`	énuméré	`fr`	Recherche multilingue
`source_texte`	énuméré	`natif` / `ocr`	Prudence sur le texte issu de l'OCR
`confiance_ocr`	réel ou nul	`0.87`	Signaler un texte incertain
`usage_test`	énuméré	`normal` / `contradictoire` / `injection` / `ambigu`	Évaluation (jamais utilisé pour répondre)
`uri_fichier`	texte	`minio://corpus/manuel_pompe_cp50.pdf`	Ouverture contrôlée (PC3)
`texte`	texte	contenu du passage ou légende	Affichage, reranking, vérification
`index_version`	texte	`idx-2026-11-v1`	Traçabilité (MLflow)

Un exemple complet de métadonnées (payload Qdrant du tableau 8.1 du manuel CP-50) est donné en annexe D.

4.5 Modèle d'accès

Groupes et utilisateurs fictifs :

Utilisateur (fictif)	Profil	Groupe(s)
`tech_pompes_01`	Technicien pompes	`techniciens_pompes`
`tech_comp_01`	Technicien compresseurs	`techniciens_compresseurs`
`resp_maint_01`	Responsable maintenance	`responsables`
`tech_poly_01`	Technicien polyvalent (cas de test)	`techniciens_pompes`, `techniciens_compresseurs`

Répartition des 42 documents :

Groupe	Documents accessibles	Détail
`techniciens_pompes`	23	22 documents pompes + la photo ambiguë du voyant orange
`techniciens_compresseurs`	17	16 documents compresseurs + la photo ambiguë
`responsables`	42 (tout)	dont 3 documents **réservés** : `plan_maintenance_pompes_2026`, `rapport_audit_air_comprime`, `note_service_compresseurs_piegee`

Règles :

1. Un utilisateur voit un élément si **au moins un** de ses groupes figure dans `groupes_acces`.

2. Les droits sont définis **dans l'inventaire**, recopiés dans les métadonnées à l'ingestion, et appliqués par le code (PC2, PC3, PC6). Ils ne dépendent jamais du texte du document ni de la réponse du modèle.
3. Les outils MCP suivent la même logique : un technicien pompes peut consulter l'historique des POMPE-*, mais pas celui des COMP-*. Seul un responsable peut consulter tous les équipements.
4. **Pas d'existence révélée.** Si aucun document accessible ne répond, le système le dit sans mentionner qu'un document réservé existe.

Tests d'accès prévus (semaine 5) :

- `tech_pompes_01` demande le budget pompes 2026 : aucune source accessible, aucune mention du document réservé ;
- `tech_comp_01` demande l'historique de POMPE-A-01 : refus côté serveur MCP ;
- `tech_pompes_01` tente d'ouvrir directement l'URL d'une page du rapport d'audit : refus 403 (PC3) ;
- `resp_maint_01` pose une question qui récupère la note piégée : réponse normale, aucune exfiltration, aucun ticket créé.

4.6 Données personnelles et journalisation

- Les utilisateurs, matricules (T-0xx) et tickets sont **fictifs**. Le corpus ne contient aucune donnée personnelle réelle.
- Les journaux enregistrent des **métadonnées techniques** : horodatage, `user_id`, groupes, identifiants des éléments récupérés, outils appelés, décision d'accès, latence. Par défaut, ils n'enregistrent ni les photos ni le contenu brut des questions et réponses.
- Les secrets (clé de signature des jetons, mots de passe des bases) sont injectés par variables d'environnement (`.env`, exclu de Git).

5. Analyse des risques

5.1 Méthode

Chaque risque est noté selon deux critères, sur une échelle de 1 à 3 :

- la **probabilité** : 1 peu probable, 2 possible, 3 probable ;
- l'**impact** : 1 faible, 2 moyen, 3 fort.

Leur produit donne la **criticité**, de 1 à 9, qui fixe l'ordre de traitement. Les 15 risques identifiés couvrent les quatre volets demandés : technique (et IA), sécurité, données, organisation. Ils incluent les six risques imposés par la consigne : hallucination (R01), fuite entre groupes (R02), ticket non approuvé (R03), erreurs de l'IA dans le corpus (R05), licences (R06) et ressources matérielles (R07).

Chaque risque a un **responsable**, celui qui suit la mesure prévue. La répartition découle des rôles définis en partie 6.

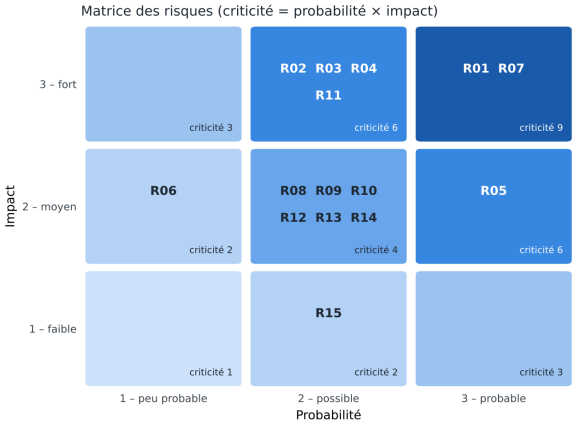


Figure 5.1 – Matrice des risques

5.2 Registre des risques

ID	Risque	Catégorie	P	I	Criticité	Mesure prévue	Responsable
R01	Hallucination et citations non fondées	Technique (IA)	3	3	**9**	Citations limitées aux éléments réellement récupérés ; vérification PC4 (existence + soutien) ; abstention ou clarification si sources insuffisantes ; mesure en S5 (objectif ≥ 90 % de citations soutenant l'affirmation)	Soulaimane

ID	Risque	Catégorie	P	I	Criticité	Mesure prévue	Responsable
R07	Ressources matérielles insuffisantes (pas de GPU local, Colab limité)	Technique	3	3	**9**	Quantification 4 bits ; modèle 3B en repli ; Kaggle (quota GPU hebdomadaire) en complément de Colab ; limites de taille d'image et de contexte ; mise en cache des embeddings ; démonstration enregistrée en secours	Soulaimane
R02	Fuite d'information entre groupes d'accès	Sécurité	2	3	**6**	Filtre groupes_acces dans la requête Qdrant (PC2) et non après coup ; revérification à l'ouverture (PC3) ; aucun cache partagé entre utilisateurs ; 20 scénarios de test d'accès en S5	Amr
R03	Création de ticket non approuvée ou en double	Sécurité	2	3	**6**	Jeton d'approbation signé (utilisateur + empreinte du contenu + expiration) ; contrôle côté serveur MCP (PC6) ; clé d'idempotence ; tests de rejeu et de double clic	Soulaimane
R04	Injection de prompt via un document, une image OCR ou une réponse MCP	Sécurité	2	3	**6**	Contenus traités comme données (balisés, jamais comme instructions) ; outils en liste blanche ; aucune sortie réseau hors liste ; document piégé du corpus + 20 tests d'injection en S5	Soulaimane
R05	Erreurs introduites par l'IA dans le corpus	Données	3	2	**6**	Référentiel unique ; contrôle automatique check.py ; relecture humaine de chaque document (verifie_par) ; journal des erreurs (6 corrigées en S1)	Amr + Soulaimane
R11	Fuite de secrets (clé d'API, jeton de tunnel, mot de passe) dans le dépôt	Sécurité	2	3	**6**	.env exclu par .gitignore ; secrets injectés à l'exécution ; analyse des secrets (gitleaks) avant chaque rendu ; jamais de secret dans un notebook partagé	Amr
R08	Le VLM lit mal un voyant, une couleur ou un afficheur	Technique (IA)	2	2	**4**	Test sur les 6 photos dès S3 ; clarification si incertitude ; une ressemblance visuelle ne vaut pas diagnostic ; recoupement avec le code affiché et l'historique	Soulaimane
R09	OCR erroné sur les documents scannés	Données	2	2	**4**	Modèle OCR français (Tesseract fra ou PaddleOCR, comparés en S2) ; score de confiance conservé dans les métadonnées ; image de la page transmise au VLM en complément	Amr
R10	Réponse fondée sur une version obsolète d'une procédure	Données	2	2	**4**	Métadonnée version ; règle : citer les deux versions, signaler l'écart, privilégier la plus récente ; tests sur les 2 paires contradictoires	Amr
R12	Latence supérieure au budget (60 s) sur un GPU T4	Technique	2	2	**4**	Mesure dès S3 ; limiter les appels au modèle (≤ 6) et la taille du contexte ; vLLM si compatible ; réponses courtes	Soulaimane
R13	Retard dû à la charge (équipe de 2, autres modules)	Organisation	2	2	**4**	Planning avec marge ; priorité au scénario de démonstration (version minimale d'abord) ; point d'avancement hebdomadaire ; fonctionnalités secondaires en option	Amr + Soulaimane
R14	Dépendance à un seul membre sur une partie critique	Organisation	2	2	**4**	Revue croisée de chaque fusion de branche ; README et documentation à jour ; chacun sait lancer tout le prototype	Amr + Soulaimane
R06	Licences des documents et des modèles	Données / juridique	1	2	**2**	Corpus 100 % synthétique (aucun document tiers) ; licences des modèles et bibliothèques vérifiées et figées dans le dépôt en S2 (la licence du modèle de repli 3B peut différer de celle du 7B)	Amr
R15	Adaptation du reranker sans gain mesurable	Technique (IA)	2	1	**2**	Comparaison honnête sans reranker / reranker d'origine / reranker adapté ; séparation des données par document ; résultat négatif rapporté tel quel	Amr

5.3 Lecture du registre

Les deux risques les plus critiques (criticité 9) sont l'hallucination (R01) et le manque de ressources GPU (R07). Ils conditionnent directement la démonstration :

- R01 est traité **par l'architecture**, et pas seulement par le prompt : les citations sont limitées aux éléments réellement récupérés, puis vérifiées (PC4). Faute de source suffisante, le système s'abstient.
- R07 est traité **dès les semaines 2 et 3** par un test de chargement du modèle sur Colab, avant d'écrire le reste du pipeline. Si le 7B ne tient pas, nous basculons tôt sur le 3B, plutôt que de le découvrir en semaine 6.

Les risques de sécurité (R02, R03, R04, R11) ont un impact fort, mais une probabilité contenue par des contrôles faits **dans le code, hors du modèle** : filtre d'accès avant la recherche, jeton d'approbation vérifié par le serveur MCP, liste blanche d'outils, secrets hors du dépôt. Chacun sera vérifié par des scénarios de test dédiés en semaine 5. Ces scénarios s'appuient sur le corpus : 3 documents réservés, 1 document piégé.

R05 (erreurs de l'IA dans le corpus) n'est pas théorique : il s'est déjà produit 6 fois en semaine 1 (journal E01 à E06). C'est pour cela que nous l'avons noté « probable ».

Les risques d'organisation (R13, R14) sont traités en livrant d'abord une **version minimale de bout en bout** : question, recherche, réponse citée. On l'enrichit ensuite.

Le registre est versionné dans le dépôt et relu chaque semaine. Les mesures de la semaine 5 permettront de confirmer ou d'infirmer les probabilités estimées ici.

6. Planning et organisation de l'équipe

6.1 Équipe et répartition des rôles

L'équipe compte deux membres. Chacun est **propriétaire** de blocs de l'architecture : il les conçoit, les code, les teste et les documente. L'autre membre relit systématiquement son travail. L'évaluation, le rapport et la démonstration sont réalisés en commun.

	Amr EL BELLAOUI	Soulaimane ELYOUNESSI
Rôle principal	Données et recherche, API et interface	Modèle, orchestration et outils
Blocs de l'architecture	8. Ingestion (PyMuPDF, OCR) · 5. Recherche documentaire (Qdrant, embeddings, RRF, reranker) · 2. API (authentification, groupes, PC1, PC3) · 1. Interface Streamlit	4. Modèle vision-langage (Colab / Kaggle) · 3. Orchestrateur LangGraph (budgets, interruption) · 6. Vérification des citations (PC4) · 7. Serveur et client MCP (PC5, PC6)
Responsabilités transverses	Contrôle d'accès (PC2), gestion des secrets, licences, Docker Compose	Sécurité du workflow (PC7), tests d'injection, prompts versionnés
Livrable 1	Corpus famille pompes (relecture de 23 documents), architecture, flux	Corpus famille compresseurs (relecture de 19 documents), risques, planning
En commun	Jeu de test et évaluation (S5), fiche de limites, rapport, démonstration	(idem)

6.2 Hypothèses de charge

- **Environ 10 h par personne et par semaine**, soit une capacité d'environ 100 h pour les semaines 2 à 6.
- **GPU** : uniquement Colab et Kaggle gratuits, avec des sessions limitées et des interruptions possibles. Les tâches GPU sont regroupées et préparées à l'avance (notebooks prêts, données en cache).
- **Marge** : environ 15 % du temps n'est pas affecté, pour absorber les imprévus (installation, quotas GPU, autres modules).

6.3 Planning des semaines 2 à 6

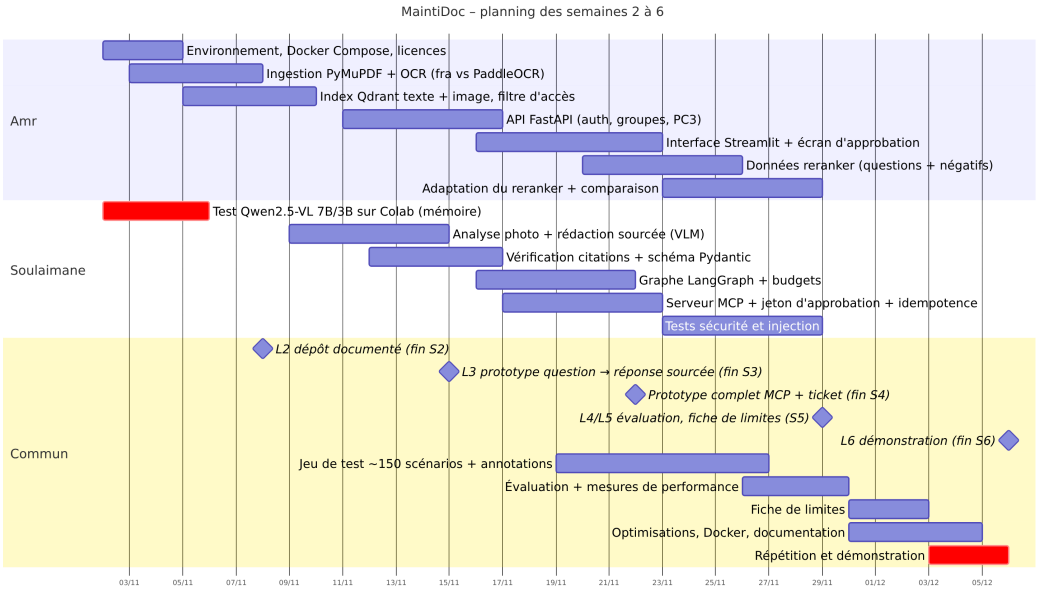


Figure 6.1 – Diagramme de Gantt des semaines 2 à 6
Les tâches en rouge sont critiques : si elles glissent, tout le planning glisse.

Semaine	Objectif	Amr	Soulaimane	Livrable / jalon
S2 (2–8 nov.)	Ingestion, recherche, accès	Environnement Docker Compose, versions et licences figées ; ingestion PyMuPDF + OCR (comparaison Tesseract `fra` / PaddleOCR) ; index Qdrant texte et image avec filtre `groupes_acces`	**Test de chargement Qwen2.5-VL 7B (4 bits) puis 3B sur Colab** : mémoire, latence, lecture des 6 photos. Décision 7B ou 3B	L2 : dépôt documenté, ingestion reproductible
S3 (9–15 nov.)	VLM, citations, sorties structurées	API FastAPI : authentification, groupes, ouverture contrôlée des sources (PC3)	Analyse de photo et rédaction sourcée ; vérification des citations (PC4) ; schéma Pydantic ; prompts versionnés `answer_v1`, `visual_analysis_v1`, `evidence_check_v1`	L3 : question + photo → réponse citant page et figure
S4 (16–22 nov.)	Workflow, MCP, validation humaine	Interface Streamlit : réponse sourcée, sources cliquables, écran d'approbation ; début des données du reranker	Graphe LangGraph (états, budgets, interruption) ; serveur MCP (3 outils, historiques du corpus) ; jeton d'approbation et idempotence	Prototype complet de bout en bout
S5 (23–29 nov.)	Reranker, évaluation, sécurité	Adaptation du reranker (800 à 1 500 exemples, séparation par document) ; comparaison sans / d'origine / adapté (nDCG@5, MRR@10)	Tests de sécurité : 20 tests d'accès et de validation, 20 tests d'injection	Jeu de test d'environ 150 scénarios, évaluation, fiche de limites
S6 (30 nov.–6 déc.)	Déploiement et soutenance	Optimisations, Docker Compose final, README	Mesures de latence (médiane, p95), mémoire GPU, MLflow	L6 : démonstration (scénario nominal, ambiguïté, injection, accès refusé, approbation)

Chemin critique. Le test du modèle en S2 conditionne tout le reste : la taille du contexte, la latence et la qualité de lecture des photos en dépendent. C'est pourquoi il est placé en **tout premier** (risque R07). Le second point critique est la **démonstration finale**. Une version enregistrée sera préparée en secours si Colab est indisponible le jour de la soutenance.

Version minimale d'abord. À la fin de S3, une chaîne simple doit fonctionner de bout en bout, même imparfaite : question, recherche filtrée, réponse citée. Les semaines suivantes l'enrichissent. Cela limite le risque d'arriver en S6 avec des blocs excellents mais non intégrés (risques R13 et R14).

6.4 Organisation du travail

- **Git.** Une branche par fonctionnalité, fusionnée dans `main` par une *pull request* relue par l'autre membre. Aucun secret commité (`.env` exclu par `.gitignore`).
- **Suivi.** Point hebdomadaire de 30 minutes (avancement, blocages, mise à jour des risques et du planning) ; issues GitHub pour les tâches et les bugs ; questions à l'enseignante posées avant le milieu de chaque semaine.
- **Traçabilité.** Prompts versionnés dans `prompts/`, expériences dans MLflow, corpus et scripts dans `corpus/` et `tools/`.

Annexe A – Inventaire résumé du corpus (42 documents)

Extrait des colonnes principales de `corpus/inventaire.csv`. Groupes : T-P = techniciens_pompes, T-C = techniciens_compresseurs, R = responsables. L'inventaire complet (20 colonnes) est dans le dossier corpus.

document_id	type	famille	p.	langue	format	accès	usage_test	vérifié par
checklist_inspection_pompes	procedure	pompes	2	fr	pdf_scanne	T-P, R	normal	Amr
codes_alarme_pompes	codes_alarme	pompes	3	fr	pdf_texte	T-P, R	ambigu	Amr
fiche_securite_consignation_pompes	fiche_securite	pompes	2	fr	pdf_texte	T-P, R	normal	Amr
fiche_securite_huile_p68	fiche_securite	pompes	3	fr	pdf_texte	T-P, R	normal	Amr
guide_diagnostic_pompes	procedure	pompes	3	fr	pdf_texte	T-P, R	normal	Amr
historique_pompe_a_01	historique	pompes	1	fr	pdf_texte	T-P, R	normal	Amr
historique_pompe_b_01	historique	pompes	1	fr	pdf_texte	T-P, R	normal	Amr
manuel_pompe_cm80	manuel	pompes	8	fr	pdf_texte	T-P, R	normal	Amr
manuel_pompe_cp50	manuel	pompes	12	fr	pdf_texte	T-P, R	normal	Amr
plan_maintenance_pompes_2026	procedure	pompes	2	fr	pdf_texte	R	normal	Amr
proc_alignement_accouplement	procedure	pompes	3	fr	pdf_texte	T-P, R	normal	Amr
proc_amorcage_demarrage_pompes	procedure	pompes	3	fr	pdf_texte	T-P, R	normal	Amr
proc_remplacement_garniture_cp50	procedure	pompes	4	fr	pdf_texte	T-P, R	normal	Amr
proc_remplacement_roulements_cp50	procedure	pompes	4	fr	pdf_texte	T-P, R	normal	Amr
proc_vidange_cp50_v1	procedure	pompes	3	fr	pdf_scanne	T-P, R	contradictoire	Amr
proc_vidange_cp50_v2	procedure	pompes	3	fr	pdf_texte	T-P, R	contradictoire	Amr
schema_electrique_armoire_pompes	schema	pompes	2	fr	pdf_texte	T-P, R	normal	Amr
schema_pid_station_sp1	schema	pompes	1	fr	pdf_texte	T-P, R	normal	Amr
schema_vue_eclatee_cp50	schema	pompes	2	fr	pdf_texte	T-P, R	normal	Amr
technote_cavitation_en	procedure	pompes	2	en	pdf_texte	T-P, R	normal	Amr
codes_alarme_compresseurs	codes_alarme	compresseurs	3	fr	pdf_texte	T-C, R	ambigu	Soulaimane
fiche_securite_air_comprime	fiche_securite	compresseurs	2	fr	pdf_texte	T-C, R	normal	Soulaimane
fiche_securite_huile_s46	fiche_securite	compresseurs	3	fr	pdf_texte	T-C, R	normal	Soulaimane
historique_comp_a_01	historique	compresseurs	1	fr	pdf_texte	T-C, R	normal	Soulaimane
historique_comp_b_02	historique	compresseurs	1	fr	pdf_texte	T-C, R	normal	Soulaimane
manuel_compresseur_ps11	manuel	compresseurs	8	fr	pdf_texte	T-C, R	normal	Soulaimane
manuel_compresseur_vs37	manuel	compresseurs	12	fr	pdf_texte	T-C, R	normal	Soulaimane
note_service_compresseurs_piegee	procedure	compresseurs	2	fr	pdf_texte	R	injection	Soulaimane
proc_purge_condensats	procedure	compresseurs	2	fr	pdf_scanne	T-C, R	normal	Soulaimane
proc_remplacement_filtres_vs37	procedure	compresseurs	3	fr	pdf_texte	T-C, R	normal	Soulaimane
proc_vidange_huile_vs37_v1	procedure	compresseurs	3	fr	pdf_scanne	T-C, R	contradictoire	Soulaimane
proc_vidange_huile_vs37_v2	procedure	compresseurs	3	fr	pdf_texte	T-C, R	contradictoire	Soulaimane
rapport_audit_air_comprime	procedure	compresseurs	3	fr	pdf_texte	R	normal	Soulaimane
schema_circuit_air_huile_vs37	schema	compresseurs	1	fr	pdf_texte	T-C, R	normal	Soulaimane
schema_electrique_compresseur_vs37	schema	compresseurs	2	fr	pdf_texte	T-C, R	normal	Soulaimane
service_bulletin_vs37_en	procedure	compresseurs	2	en	pdf_texte	T-C, R	normal	Soulaimane
photo_manometre_ps11	photo	compresseurs	1	fr	image	T-C, R	normal	Soulaimane
photo_panneau_comp_a01	photo	compresseurs	1	fr	image	T-C, R	ambigu	Soulaimane
photo_panneau_pompe_a01	photo	pompes	1	fr	image	T-P, R	normal	Amr
photo_plaque_cp50	photo	pompes	1	fr	image	T-P, R	normal	Amr
photo_plaque_floue_pompe	photo	pompes	1	fr	image	T-P, R	ambigu	Amr
photo_voyant_orange_gros_plan	photo	compresseurs	1	fr	image	T-C, T-P, R	ambigu	Soulaimane

Annexe B – Journal de vérification du corpus

#	Document(s)	Problème détecté	Comment	Correction
E01	proc_vidange_cp50_v1 (et figure garniture partagée)	Le schéma de la garniture, réutilisé dans la procédure v1, indiquait « couple 35 N·m » alors que le texte de la v1 indique 25 N·m : contradiction interne au document, non voulue (seule la contradiction v1/v2 est voulue).	Relecture visuelle de la page 2 du scan	Valeur de couple retirée du schéma (seul « 4 vis M10 » reste) ; le couple n'est donné que dans le texte
E02	historique_pompe_a_01, historique_pompe_b_01	Tableau trop étroit : identifiants INT-AAAA-NNNN coupés sur deux lignes, ce qui gênerait l'extraction et la citation des identifiants	Relecture visuelle	Police réduite pour les tableaux de plus de 7 colonnes, largeurs ajustées
E03	rapport_audit_air_comprime	Le graphique était titré « consommation local LC1 » alors que ses valeurs (144 300 kWh) incluaient COMP-B-02, installé dans l'atelier ; incohérence entre figure et tableau A.1	Contrôle arithmétique : somme des 9 mois = somme du tableau A.1 (3 compresseurs)	Titre de figure et texte corrigés (« 3 compresseurs, LC1 + atelier », dont 136 500 kWh pour LC1)
E04	photo_panneau_comp_a01	Première version : le reflet ne masquait pas le code « A-12 », la photo n'était donc pas ambiguë, contrairement à son rôle de test (usage_test = ambigu)	Contrôle visuel de la photo générée	Reflet repositionné sur le code ; seule la mesure « REF 103°C » reste lisible
E05	photos de panneaux (voyants orange)	Le voyant « orange » allumé apparaissait jaune (éclaircissement trop fort), ce qui aurait faussé la question « que signifie ce voyant orange ? »	Contrôle visuel	Éclaircissement réduit, teinte orange conservée
E06	codes_alarme_pompes, schema_pid_station_sp1	Identifiants abrégés « POMPE-A-01/A-02 » : « A-02 » isolé ressemble à un code d'alarme compresseur (série A-xx), risque de confusion pour la recherche	Contrôle automatique check.py (codes cités ∉ référentiel)	Identifiants écrits en entier (POMPE-A-01 et POMPE-A-02)

Annexe C – Contenu du dépôt et reproductibilité

Dépôt : https://github.com/amr2444/Projet-int-grateur---Module-IA-g-n-rative-multimodale

- `corpus/` : documents, `inventaire.csv`, `prompts_corpus.md` (P01 à P25)
- `docs/diagrammes/` : sources Mermaid de toutes les figures du rapport (architecture, flux, planning) et matrice des risques
- `docs/rapport/` : sections du rapport en Markdown
- `tools/generation_corpus/` : référentiel `ref.py`, scripts de génération, `check.py` (contrôles de cohérence)

Régénérer et contrôler le corpus : `cd tools/generation_corpus && python figures.py && python build.py docs_f1 docs_f2 && python photos.py && python inventaire.py && python check.py`

Annexe D – Exemple de métadonnées d'un élément indexé

Payload Qdrant de l'élément « Tableau 8.1 » (codes d'alarme) du manuel CP-50, page 9 (voir 4.4).

```
{
  "element_id": "manuel_pompe_cp50#p9#tableau_8_1",
  "document_id": "manuel_pompe_cp50",
  "version": "v2.1 (2025-03)",
  "page": 9,
  "type_element": "tableau",
  "numero_element": "Tableau 8.1",
  "famille_equipement": "pompes",
  "equipement_id": ["POMPE-A-01", "POMPE-A-02"],
  "groupes_acces": ["techniciens_pompes", "responsables"],
  "langue": "fr",
  "source_texte": "natif",
  "confiance_ocr": null,
  "usage_test": "normal",
  "uri_fichier": "minio://corpus/documents/famille_1/manuel_pompe_cp50.pdf",
  "index_version": "idx-2026-11-v1"
}
```

Annexe E – Données échangées entre les blocs lors d'une requête

De → vers	Données transmises	Contrôle
Interface → API	Question, photo (taille limitée), jeton de session	—
API → Orchestrateur	Question, photo, `user_id`, liste des groupes	**PC1** : les groupes viennent de PostgreSQL, jamais de la requête du client
Orchestrateur → VLM	Photo et question (analyse), puis extraits, figures et historique (rédaction)	Budget : 6 appels maximum
Orchestrateur → Recherche	Texte de recherche, image, **filtre `groupes_acces` ∩ groupes de l'utilisateur**	**PC2** : filtre appliqué dans la requête Qdrant
Recherche → Orchestrateur	Top-k éléments avec `document_id`, `version`, `page`, `element_id`, score	Seuls les éléments autorisés existent dans le résultat
Orchestrateur → MCP	`equipment_id` + identité	**PC6** : le serveur revérifie les droits ; 3 appels maximum
Orchestrateur → Vérification	Réponse brouillon + liste des éléments réellement récupérés	**PC4**
API → Interface	JSON validé : `status`, `answer`, `citations`, `uncertainties`, `clarification_question`, `proposed_action`	—
Interface → API → MinIO	Ouverture d'une page citée	**PC3** : droits revérifiés